Threads

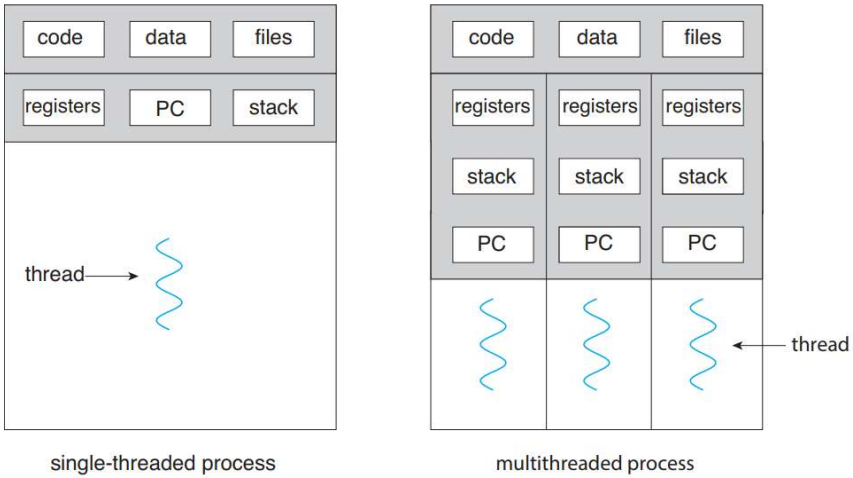
Overview

- **What is a Thread?**
  - A **thread**, or lightweight process, is a basic unit of CPU utilization
  - It is a single sequential flow of control within a process.

- **Multithreading:** A **multithreaded process** can have multiple threads of control
  - As an example, a word processor might have one thread to display graphics, another to read input, and a third to perform spelling and grammar checks in the background.

- **Thread vs. Process**
  - A process is a heavyweight entity that includes a program's code, data, and resources, while a thread is a lightweight flow of execution
  - Threads within the same process share the process's code, data, and resources.

- **Why Use Threads?**
  - Threads are used to enable an application to perform multiple tasks concurrently, improving responsiveness, resource sharing, and efficiency on multicore systems



single-threaded process

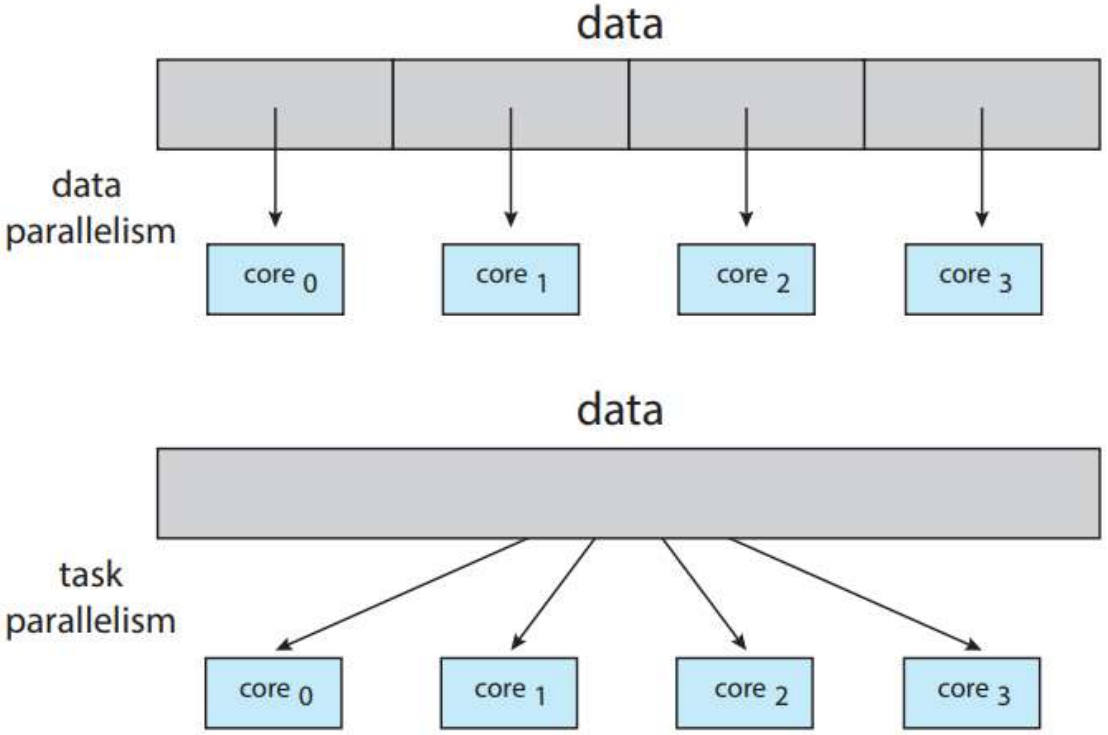
multithreaded process

Multicore Programming

- **Multicore vs. Multiprocessor**
- A **multicore system** has multiple processing cores on a single physical chip, while a **multiprocessor system** has multiple physical chips, each with one or more cores
  - **Parallelism:** Parallelism implies a system can perform more than one task simultaneously
  - **Data Parallelism:** Distributes subsets of the same data across multiple cores, each performing the same operation
  - **Task Parallelism:** Distributes different tasks to different cores

- **Challenges of Multicore Programming:**
  - **Dividing Activities:** How do you break a task into multiple, concurrent subtasks?
  - **Balancing:** How do you ensure that the subtasks perform an equal amount of work to maximize efficiency?
  - **Data Splitting:** How do you split the data accessed by the subtasks?
  - **Synchronization:** How do you coordinate threads so they don't interfere with each other?

Data and Task Parallelism



Amdahl's Law

- Formula identifies the potential performance gains from adding cores to an application
- It states that as the number of cores increases, the performance gain is limited by the sequential portion of the program.

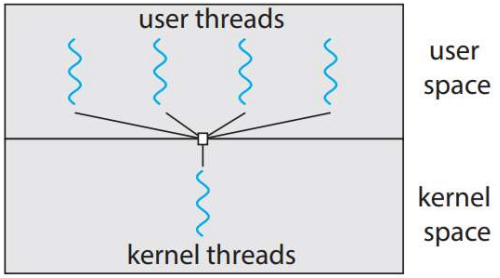
$$speedup \leq \frac{1}{S + \frac{(1-S)}{N}}$$

where:
S – sequential portion
N – number of processors

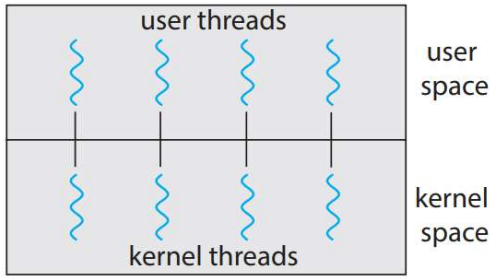
Multi-Threading Models

- **Types of Threads:**
  - **User-Level Threads:** Threads are managed by a user-level library without kernel support
  - **Kernel-Level Threads:** Threads are managed by the operating system kernel

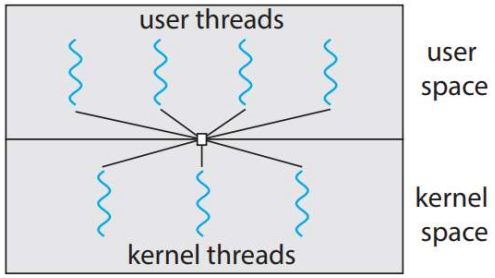
- **Mapping Models:**
  - **Many-to-One Model:** Maps many user-level threads to a single kernel thread
    - This is efficient but if one thread makes a blocking system call, the entire process is blocked
  - **One-to-One Model:** Maps each user-level thread to a kernel thread
    - This allows for true concurrency on multicore systems and avoids blocking the entire process
  - **Many-to-Many Model:** A flexible model where many user-level threads are mapped to a smaller or equal number of kernel threads
    - The OS can create as many kernel threads as needed, providing a good balance of efficiency and concurrency.



Many-to-one



One-to-one



Many-to-many

Thread Libraries

- A **thread library** provides the programmer with an API for creating, managing, and synchronizing threads
- There are two primary ways to implement a library:
  - **User-Level Library:** Provides no kernel support, and all thread management happens at the user level
  - **Kernel-Level Library:** Provides a direct API for the operating system's kernel threads
- **Common Thread Libraries:**
  - **POSIX Pthreads:** A widely used, platform-independent API for thread creation and synchronization
    - It is a user-level library that can be used on many operating systems
  - **Win32 Threads:** A kernel-level thread library provided by the Windows operating system
  - **Java Threads:** Managed by the Java Virtual Machine (JVM).
    - The JVM typically uses the underlying OS's threading model

Implicit Threading

- Implicit threading is a shift in programming where the creation and management of threads are handled by compilers and run-time libraries, rather than explicitly by the programmer
- **Key Approaches:**
  - **Thread Pools:** A collection of pre-created threads that are ready to perform tasks
    - When a task arrives, a thread from the pool is assigned to it
    - This saves the overhead of repeatedly creating and destroying threads
  - **OpenMP:** A set of compiler directives and an API for parallel programming in C, C++, and Fortran
    - It instructs the compiler to generate multithreaded code
  - **Grand Central Dispatch (GCD):** A technology for Apple's macOS and iOS that allows developers to manage concurrent execution by placing tasks into dispatch queues
  - **Intel Threading Building Blocks (TBB):** A library that simplifies parallel programming for multicore processors by focusing on task-based parallelism

Issues in Threading

- **The fork() and exec() System Calls:** The behavior of fork() can be problematic in a multithreaded program
  - Does fork() duplicate all threads or only the one that called the fork()?
  - Most systems provide a mechanism to decide
- **Signal Handling:** A **signal** notifies a process that an event has occurred
  - In a multithreaded program, it's not always clear where the signal should be delivered (to which thread)
- **Thread Cancellation: Thread cancellation** is the termination of a thread before it has completed
- This can be done in two ways:
  - **Asynchronous Cancellation:** Terminates the target thread immediately
  - **Deferred Cancellation:** Allows the target thread to periodically check if it should terminate, providing a chance to clean up resources before exiting
- **Thread-Local Storage (TLS):** This allows each thread to have its own copy of data that would normally be shared
  - This is useful for when each thread needs its own data but there is no need for a process-wide variable.

Operating System Examples

- **Windows Threads:** A **one-to-one mapping** model.
  - The Windows API provides a comprehensive library for creating, synchronizing, and managing threads
- **Linux Threads:** Linux refers to threads as **tasks**
  - The Linux kernel does not have a separate thread concept; instead, it provides a clone() system call that creates a process
  - This technique shares the address space of the parent, effectively creating a thread
- **Java Threads:** The JVM manages Java threads
  - The JVM's implementation can vary, but it often uses the underlying OS's **one-to-one mapping model**

Threads